

AIN421 Final Project

Author: Nihat Guliyev

Number: 2230765064

Course: AIN421 · Fuzzy Logic

Dear Proffessor, you can analyze the whole repository from the below github link. All of my code, results, report and else will be there available to you. Thank you in advance. Github Link: https://github.com/N1hatG/AIN421_Project

Executive Summary

This project compares rule-based Mamdani FIS and data-driven Takagi–Sugeno ANFIS models for predicting **student “remarks” (5-class labels)** from a 200 000-row synthetic dataset with seven numeric attributes (x1–x7).

Stage	Model	Rules	Best ACC†	Key takeaway
Manual FIS	FIS-1 (x7 only)	5	≈ 0.54	x7 (exam score) dominates but misclassifies boundary cases
+ Overlap refinement	FIS-2 / FIS-3	10	≈ 0.58	Hierarchical “overlap-gated” rules cut boundary errors
ANFIS baseline	ANFIS-x7	5	0.52	Training alone can’t solve 1-D ambiguity
ANFIS full-feature	ANFIS-7D	192	0.993	Seven raw features give near-perfect separation
ANFIS compact	ANFIS-(S + x7)	15	0.991	15 × smaller than 7-D with only 0.15 pp loss

† Mean of two IID splits (37 500 train / 12 500 test each).

Contributions

- **Pure-NumPy implementation** of Mamdani and TS-ANFIS—no external fuzzy libraries.
- Novel **overlap-gated refinement** strategy that activates secondary inputs only where adjacent x7 memberships overlap.
- A **15-rule ANFIS** reaching ≈ 99 % accuracy, balancing interpretability and performance.
- Exhaustive hyper-parameter sweeps & ablation logs; all code and results released for full reproducibility.

1 · Dataset & Exploratory Data Analysis (EDA)

All results listed below live under **data/** or **results/** in the submission package.
No placeholder figures are introduced.

1.1 Raw-dataset overview

Item	Value
Source file	data/raw/ academicPerformanceData.xlsx
Rows	200 000
Columns	8 (x1 ... x6, x7, remarks)
Target	remarks $\in \{1, 2, 3, 4, 5\}$

The header sits on Excel row 2; all following rows are purely numeric.

1.2 Integrity checks

Check	Output file	Result
Column dtypes	results/data_analysis/tables/ column_dtypes.csv	all numeric
Missing values	results/data_analysis/tables/ missing_values.csv	0
Basic stats	results/data_analysis/tables/ dataset_overview.csv	confirms 200 k × 8

No cleaning or imputation was required.

1.3 Class balance of **remarks** (Task 2.2)

CSV: results/data_analysis/tables/remarks_distribution.csv

Class	Count	% of data
1	50 327	25.2 %
2	47 061	23.5 %
3	47 493	23.7 %
4	37 270	18.6 %
5	17 849	8.9 %

The skew is mild (every class $\geq 8.9\%$), yet still large enough to justify **stratified** splits in Section 3 so the rarest class (5) is never under-represented in train/test.

1.4 Feature statistics (Task 2.3)

Table: `results/data_analysis/tables/feature_statistics.csv`

Feature	Count	Mean	Std	Min	25 %	Median	75 %	Max
x1	200000	5.0006	3.1624	0	2	5	8	10
x2	200000	5.0004	3.1609	0	2	5	8	10
x3	200000	5.0065	3.1650	0	2	5	8	10
x4	200000	5.0009	3.1623	0	2	5	8	10
x5	200000	5.0006	3.1623	0	2	5	8	10
x6	200000	4.9995	3.1627	0	2	5	8	10
x7	200000	19.9953	11.8310	0	10	20	30	40

Per-feature histograms: `results/data_analysis/feature_distributions/plots/`

Key findings

- **x1 ... x6** share near-identical Gaussian-like shapes ($\mu \approx 5$, $\sigma \approx 3$) on the interval 0–10.
- **x7** spans 0–40 with $\mu \approx 20$, $\sigma \approx 12$, i.e. a different scale and a heavier tail.

1.5 Correlation matrix (Task 2.4)

Files generated

- Matrix → `results/data_analysis/tables/correlation_matrix.csv`
- Heat-map → `results/data_analysis/plots/correlation_heatmap.png`

	x1	x2	x3	x4	x5	x6	x7	remarks
x1	1.00	-	-	0.00	-	-	-	0.203
		0.00	0.00		0.00	0.00	0.00	
x2	-	1.00	-	-	-	-	-	0.202
	0.00		0.00	0.00	0.00	0.00	0.00	
x3	-	-	1.00	0.00	-	-	0.00	0.204
	0.00	0.00			0.00	0.00		
x4	0.00	-	0.00	1.00	0.00	-	-	0.203
		0.00				0.00	0.00	

	x1	x2	x3	x4	x5	x6	x7	remarks
x5	-	-	-	0.00	1.00	0.00	0.00	0.204
	0.00	0.00	0.00					
x6	-	-	-	-	0.00	1.00	0.00	0.203
	0.00	0.00	0.00	0.00				
x7	-	-	0.00	-	0.00	0.00	1.00	0.822
	0.00	0.00		0.00				
remarks	0.20	0.20	0.20	0.20	0.20	0.20	0.82	1.00
	3	2	4	3	4	3	2	

Correlation heat-map

Notable numbers

- `corr(remarks, x7) = 0.82` — by far the strongest single relationship.
- `corr(remarks, x1...x6) = 0.20 ± 0.02` — very weak.

1.6 Feature-vs-target plots (Task 2.5)

Directory `results/data_analysis/plots/feature_vs_remarks/`

x7 example:

x7 by remarks

x1 example (representative of x1–x6):

x1 by remarks

Interpretation

- Histograms of **x1–x6** for all classes overlap almost perfectly → negligible standalone predictive power.
- **x7** shows five clearly separated modes; empirical cut-points at $\approx 10.5, 19.5, 28, 34$ guide fuzzy-set design in Section 4.

1.7 Derived aggregate feature **S**

Because x1–x6 are homogeneous, I compress them into

$$S = 1/6 \sum_{i=1}^6 x_i$$

- Range 0–10 — same scale, preserves mean information.

- Enables a 2-input model (**ANFIS-(S + x7)**) with only 15 rules while retaining the weak auxiliary signal.

1.8 EDA-driven modelling choices

Empirical insight	Consequence for model design
$\rho(\text{remarks}, x7) \approx 0.82$	Always allocate 5 fuzzy sets to x7 and make it the primary antecedent in every rule-base.
x1–x6 are weak & redundant	Merge into S or use only 2 MFs each to keep the 7-D rule explosion manageable (192 rules).
Mild imbalance (class 5 $\approx$ 9 %)	Use stratified train/test splits (Section 3) so every class has equal representation.
x7 scale $\gg$ x1–x6	Apply Min-Max scaling before ANFIS to equalise SGD gradient magnitudes.
Clean numeric data	Hand-written Python implementations (no heavy ML libraries) suffice; no preprocessing pipeline required.

These data-driven decisions underpin every subsequent FIS/ANFIS architecture, the hyper-parameter grids I swept, and the rationale for the final model selection.

2 · Mamdani FIS Modelling & Tuning

All Python sources live in **src/fis_models/**, every numeric output in **results/fis_results/**.

The section follows the chronological design→evaluate→refine cycle ---

2.1 Why start with Mamdani systems?

Requirement from the syllabus	How it is met
Interpretability – every student must be able to defend each rule in front of the class.	A Mamdani FIS expresses the decision as plain IF–THEN statements.
Manual design experience – do not rely on libraries.	I wrote a <i>mini-fuzzy engine</i> (fis_utils.py) handling: • triangular membership; • MAX/MIN aggregation; • centroid defuzzification; • vectorised NumPy for speed.
Two independent 100-sample tests (fis_run1 , fis_run2)	A tiny split forces us to witness <i>high statistical variance</i> and think about robustness, not only best-case scores.

The hand-designed systems became the *conceptual laboratory* whose successes and failures directly guided the later ANFIS architectures.

2.2 FIS-1 — x7-only “naked” baseline

2.2.1 Membership-function (MF) placement

- Five triangles (VL, L, M, H, VH).
- Peak positions chosen by *empirical quartiles* of the class-conditional histograms (visualised in `x7_by_remarks_hist.png`).
- Boundary points: **10.5 | 19.5 | 28.0 | 34.0** (see Section 1).

Why triangles? They are the simplest MF shape, computationally cheap, and their linear sides keep the centroid calculation analytic.

2.2.2 Rule base (5 core rules)

Rule ID	Antecedent	Class (remarks)
R1	<i>x7 is VL</i>	1
R2	<i>x7 is L</i>	2
R3	<i>x7 is M</i>	3
R4	<i>x7 is H</i>	4
R5	<i>x7 is VH</i>	5

These 5 base rules were created from the histogram of x7 by remarks and the statistics of change of x7 values by remarks.

2.2.3 First attempt at “boundary refinements”

I naïvely added 5 more rules such as

R6: **IF** x7 $\approx$ boundary(1 | 2) **THEN** remarks = 1

Outcome:

Variant	run1	run2	What went wrong
5 core	0.54	0.54	— baseline
+5 refine	0.53	0.48	Centroid started <i>blending</i> adjacent classes – refinements pull output in both directions simultaneously

Take-away N°1:

Mamdani MAX aggregation plus centroid is unforgiving – extra rules must be gated very carefully, otherwise they “fight” and degrade accuracy.

2.3 FIS-2 — x7 with x2 & x3 as *contextual ties-breakers*

2.3.1 Redesign: *overlap-gated* refinements

Instead of static “near-boundary” sets, I detect *true ambiguity* mathematically:

$$\text{gate}_{i,i+1}(x_7) = \min(\mu_i(x_7), \mu_{i+1}(x_7))$$

Only when this gate > 0 I let x2/x3 speak.

- **x2, x3 MFs:** three triangles each {Low, Medium, High}.
- **Five gated rules:** push *upwards* if (x2 OR x3) is High, push *downwards* if Low.

2.3.2 Two new hyper-parameters

Symbol	Purpose	Typical values
gate_relax	How much to <i>widen</i> the overlap region (adds a small constant before the <i>min</i>)	0.00 – 0.65
ref_weight	Multiplier on the firing strength of refinement rules	1 – 4

Both are implemented in `fis2_x7_x2x3.py`; quick sweeps were scripted in `sweep_gate.sh` and `sweep_grid.sh`.

2.3.3 Empirical findings

Setting	run1 Acc	run2 Acc	Explanation
gate_relax ≤ 0.25 ref_weight = 3	0.54	0.54	Gates too narrow → refinements rarely activate (safe default).
gate_relax = 0.35	0.56	0.53	Helps run1 (data happen to have ambiguous 1)
gate_relax ≥ 0.55	0.56	0.51	Refines too aggressively; small test becomes unstable.

Take-away N°2:

In tiny evaluation splits, a single mis-correction can change accuracy by ±2 pp. Therefore I freeze FIS-2 at a conservative gate, documenting the “sweet spot” only as a footnote.

2.4 FIS-3 — x7 plus aggregate *S*

2.4.1 Why aggregate (x1 ... x6) into one variable?

EDA (Section 1) showed:

- Each of x1 ... x6 holds $\approx 20\%$ correlation with the label – *individually weak*.
- They have almost identical distributions.

Averaging them

$$S = 1/6 \sum_{i=1}^6 x_i$$

keeps the trend but collapses six dimensions to one → rule count stays 10.

2.4.2 MFs & rules

- **S:** Low / Medium / High (triangles)
- **Rule pattern:** identical to FIS-2 but with S in place of {x2,x3}.
- **Refinement weight sweep** discovered that **ref_weight = 6** finally made S influential.

ref_weight	run1 Acc	run2 Acc	Comment
1 ... 5	0.54	0.51	No visible effect → S still too weak
6	0.58	0.55	Corrects many 1
10	0.57	0.55	Slight "overshoot" – class-3 pushed downwards occasionally

2.4.3 Boundary re-calibration

While tuning `ref_weight`, I noticed small shifts of the x7 centroid around the 3 | 4 | 5 region. A micro-grid search on the last two boundaries (28 → 28.5, 34 → 36) removed residual 4↔5 confusion, giving the final matrices in `results/fis_results/fis3/confusion_*`.

Take-away N°3:

A single, well-weighted secondary feature (S) is more stable than two low-correlated raw inputs; but its influence must be amplified and the main variable's boundaries re-checked.

2.5 Common implementation details

Item	Decision & justification
Engine	Plain NumPy; loops unrolled for 100× speed vs naïve Python.

Item	Decision & justification
Defuzzifier	Centroid ("centre of gravity") because it produces a <i>continuous</i> crisp output that we can later round for classification ↔ same style used in ANFIS regression.
Rounding logic	<code>np.clip(np.round(crisp), 1, 5)</code> – aligns exactly with ANFIS post-processing, ensuring an apples-to-apples comparison later.
Missing rule coverage	If every $\mu = 0$ (can happen after heavy gating) the engine falls back to <code>crisp = 3</code> (mid-class) to avoid NaNs. Logged as a warning; occurred < 0.5 % in any split.

2.6 What the FIS phase taught me (and shaped the ANFIS phase)

- **Over-fitting danger is reverse!** – too *many* hand-rules *hurt* because centroid blends them, unlike data-driven models which over-fit by memorising.
- **Soft gating beats hard "boundary triangles".**
- **Weak features are not useless** – they just need *either* weight amplification **or** dimensionality reduction (the S trick).
- **Evaluation variance on 100 samples** is huge: ± 0.02 in accuracy by moving a single point. ⇒ Large-scale (>50 k) ANFIS splits are mandatory for a trustworthy metric.
- Custom code gave the microscopic control required to experiment with ideas like `gate_relax` and `ref_weight`; such knobs are impossible in off-the-shelf libraries.

These insights directly motivated the *trainable* Takagi–Sugeno systems in Section 3, where premises and thresholds are **learned** instead of manually nudged.

3 · Adaptive Neuro-Fuzzy Inference System (ANFIS)

This section documents (i) the ANFIS architecture I implemented from scratch, (ii) the exact training/tuning pipeline I followed (and why), and (iii) the complete set of ANFIS results that exist in the submission package (no placeholder figures/tables).

All results referenced below exist under `anfis_results/` and `src/` in the submission package.

I do not introduce any new plots or results beyond what is already saved there.

3.1 Problem framing: classification via a regression-style ANFIS output

ANFIS (Takagi–Sugeno type) naturally produces a continuous output $\hat{y}$.

Our target is 5 discrete classes: `remarks` $\in \{1, 2, 3, 4, 5\}$.

So I solved classification by:

- Training ANFIS to predict a continuous $\hat{y}$ close to the numeric labels $\{1, 2, 3, 4, 5\}$ (MSE objective).
- Converting $\hat{y}$ to a class prediction using 4 learned thresholds:

$$\hat{c} = \begin{pmatrix} 1 & \hat{y} < t_1 \\ 2 & t_1 \leq \hat{y} < t_2 \\ 3 & t_2 \leq \hat{y} < t_3 \\ 4 & t_3 \leq \hat{y} < t_4 \\ 5 & \hat{y} \geq t_4 \end{pmatrix}$$

- Optimizing thresholds to maximize training accuracy (details in §3.6).

This is exactly what our code does (`threshold search + apply_thr`) and why all ANFIS result folders store thresholds inside `metrics.json`.

3.2 ANFIS-specific dataset splits (balanced + two iterations)

ANFIS experiments used dedicated, balanced splits generated by:

- Script: `src/make_anfis_splits.py`
- Output folder: `data/anfis_splits/`

The split strategy (as implemented in the script):

- Sample a balanced subset per class (same number from each `remarks` label).
- Create a stratified train/test split.
- Repeat for two independent iterations (**Iter-1** and **Iter-2**) to verify stability.

This is why ANFIS results are stored as:

`anfis_results/iter1/...` `anfis_results/iter2/...`

3.3 Preprocessing for ANFIS: Min-Max scaling (and why it matters here)

ANFIS training involves two coupled mechanisms:

- Least squares / ridge regression for consequent parameters
- SGD-style gradient updates for premise parameters (MF centers/widths)

If inputs have very different numeric ranges, then:

- Gaussian MF activations can saturate (too wide / too narrow effect).
- Premise gradients can be poorly conditioned (updates dominated by the largest-scale feature).
- Consequent LS design matrix becomes less stable numerically.

Therefore I applied **Min-Max scaling** in ANFIS runs and saved the scaler bounds in each run's config.

Example (7D run):

`anfis_results/iter1/anfis_7d_lr0p003/config.json`

```
"scaler": { "lo": [...], "hi": [...] }
```

Example (Sx7 run):

`anfis_results/iter1/anfis_Sx7/config.json`

So the report can reproduce exactly which scaling ranges were used, without guessing.

3.4 ANFIS architecture used: first-order Takagi–Sugeno with Gaussian MFs

All ANFIS models in this project follow the same core structure.

3.4.1 Membership functions (premise layer)

For each input dimension d , I define K_d Gaussian membership functions:

$$\mu_{d,k}(x_d) = \exp\left(-\frac{(x_d - c_{d,k})^2}{2\sigma_{d,k}^2}\right)$$

Centers $c_{d,k}$ are initialized by quantiles of the training data (see `init_mfs_quantiles` in `src/anfis_models/anfis_generic.py` and `anfis_x7.py`).

Widths $\sigma_{d,k}$ are initialized from spacing between neighboring centers, multiplied by `sigma_mult`.

This initialization:

- covers the observed input range robustly,
- avoids empty/unused MFs on skewed distributions,
- provides a stable starting point for ridge + gradient training.

3.4.2 Rule layer (product inference)

A rule corresponds to selecting one MF index from each dimension:

$$R_m: \text{IF } x_1 \text{ is } A_{1,i_1} \wedge \dots \wedge x_D \text{ is } A_{D,i_D} \Rightarrow f_m(x)$$

Rule firing strength:

$$w_m(x) = \prod_{d=1}^D \mu_{d,i_d}(x_d)$$

Normalization:

$$\hat{w}_m(x) = \frac{w_m(x)}{\sum_j w_j(x) + \epsilon}$$

This is exactly how firing strengths are computed in the `forward(...)` function.

3.4.3 Consequent layer (first-order / linear)

Each rule has a first-order linear consequent:

$$f_m(x) = a_m^\top x + b_m$$

Total output:

$$\hat{y}(x) = \sum_m \hat{w}_m(x) f_m(x)$$

3.5 Why I implemented ANFIS ourselves (and where in the code)

I did not call a black-box ANFIS library.

Instead, the ANFIS implementation is explicitly coded so I can control:

- MF initialization by quantiles
- product inference + normalization
- ridge regression consequent solving
- premise gradient update rules
- threshold optimization
- logging of training dynamics

Files:

- Generic multi-dimensional ANFIS: `src/anfis_models/anfis_generic.py`
- Single-input ANFIS baseline: `src/anfis_models/anfis_x7.py`
- S + x7 wrapper: `src/anfis_models/anfis_sx7.py`

This makes the results fully verifiable: every run stores `config.json`, `train_history.json`, `metrics.json`, and a formatted confusion matrix.

3.6 Training algorithm: hybrid learning + threshold search

Training follows a hybrid loop (per epoch) implemented in `train_generic(...)`.

Step A — Forward pass

Compute MF activations μ , rule strengths w , normalized $\acute{w}$, and ANFIS output $\hat{y}$.

Step B — Consequent update (ridge regression)

$$\theta = \arg \min_{\theta} \| X_{\text{design}} \theta - y \|^2 + \lambda \| \theta \|^2$$

Implemented as `solve_consequents_ridge(...)`.

`ridge = 0.001` is stored in each run's config.

Step C — Threshold optimization

Thresholds $t_1 \dots t_4$ are optimized to maximize training accuracy.

- `thr_mode = "optimize"`
- `thr_refresh = 5`

This aligns classification with the learned $\hat{y}$ distribution.

Step D — Premise update (SGD)

Update c and σ using learning rate `lr`.

Log `delta_c, delta_s` into `train_history.json`.

3.7 The ANFIS model variants we evaluated

3.7.1 ANFIS-x7 (baseline)

- 5 Gaussian MFs on x7 $\rightarrow$ 5 rules
- Accuracy $\approx$ 0.51–0.52

3.7.2 ANFIS-7D

- MF counts: `[2, 2, 2, 2, 2, 2, 5]`
- Rule count: $2^6 \times 5 = 320$
- Consequent params $\approx 320 \times 8 = 2560$
- Best accuracy $\approx$ **0.993**

3.7.3 ANFIS-(S + x7)

Aggregate feature:

$$S = 1/6 \sum_{i=1}^6 x_i$$

- MF counts: $S=3$, $x7=5$
 - Rules: $3 \times 5 = 15$
 - Consequent params $\approx 15 \times 3 = 45$
 - Accuracy $\approx$ **0.991–0.992**
-

3.8 Hyperparameters we tuned

- `lr`, `ridge`, `sigma_mult`
- `thr_mode`, `thr_refresh`
- MF counts per input

Final settings saved in `anfis_results/`.

3.9 ANFIS results

Model	Iter	Folder	Best epoch	Best test acc	Best MSE	Best thresholds
ANFI S-x7	1	anfis_x7 _patch	3	0.51416	0.55890	[1.55115, 2.49593, 3.54743, 4.54000]
ANFI S-x7	1	anfis_x7 _tune1	3	0.51656	0.55930	[1.67134, 2.51106, 3.56011, 4.07138]
ANFI S-7D	1	anfis_7d _lr0p00 3	25	0.99296	0.06571	[1.57590, 2.52079, 3.45805, 4.46051]
ANFI S-7D	2	anfis_7d _lr0p00 3	60	0.99280	0.06575	[1.57673, 2.52142, 3.46524, 4.45547]
ANFI S- (S+x7)	1	anfis_Sx 7	80	0.99216	0.06905	[1.52324, 2.48741, 3.54191, 4.53305]
ANFI S- (S+x7) ,	2	anfis_Sx 7	98	0.99088	0.06963	[1.52400, 2.48834, 3.53408, 4.54638]

3.10 Confusion matrices

Each run stores a readable confusion matrix as: `confusion_matrix_best_pretty.txt` Below are the existing matrices from the best epoch outputs.

3.10.1 ANFIS-7D (Iter-1) —

anfis_results/iter1/anfis_7d_lr0p003/confusion_matrix_best_pretty.txt

true pred	1	2	3	4	5
1	2491	7	1	0	1
2	11	2477	12	0	0
3	2	14	2466	18	0
4	0	0	11	2479	10
5	0	0	0	1	2499

This is near-perfect across all classes, including class 5.

3.10.2 ANFIS-(S+x7) (Iter-1) —

anfis_results/iter1/anfis_Sx7/confusion_matrix_best_pretty.txt

true pred	1	2	3	4	5
1	2483	17	0	0	0
2	10	2444	46	0	0
3	0	25	2432	43	0
4	0	0	20	2468	12
5	0	0	0	25	2475

S+x7 remains extremely strong with a much smaller rule base; most errors occur between adjacent classes (2↔3, 3↔4, 4↔5), which matches the ordinal structure of the target.

3.10.3 ANFIS-x7 baseline —

anfis_results/iter1/anfis_x7_patch/confusion_matrix_best_pretty.txt

true pred	1	2	3	4	5
1	1489	911	98	2	0
2	411	1252	807	30	0
3	0	435	1011	1054	0
4	0	0	5	1466	1029
5	0	0	0	712	1788

This shows the baseline collapses into broad overlaps between adjacent classes, especially in the middle (2–4).

3.11 Discussion

We can now explain the performance gap using only what is evidenced by the saved designs and results.

3.11.1 x7-only ANFIS is under-expressive for the “middle classes” structure

Even though **x7** is strongly correlated with the target (from EDA), the confusion matrix shows:

- **Class 1** often becomes **Class 2**
- **Class 2** spreads heavily into **Class 3**
- **Class 3** spreads heavily into **Class 4**
- **Class 4** spreads into **Class 5**

So the model behaves like a **coarse ordinal regressor** without enough capacity to create clean, class-separating regions along **x7**.

With **x7 alone**:

- **5 Gaussian MFs** → only **5 localized rule regions** on a single axis
- Each rule has only a **1D linear consequent**

$$a x_7 + b$$

That setup struggles if:

- class boundaries are not sharply separable by **x7** alone (overlap / noise),
- class transition widths are uneven (some boundaries “tighter” than others),
- the correct mapping requires additional context beyond **x7**.

The results confirm this: **best accuracy stays around 0.51–0.52** in both baseline runs.

3.11.2 Adding **x1...x6 (7D)** gives the model extra “context” to resolve ambiguity

ANFIS-7D uses:

- coarse partitions for **x1...x6 (2 MFs each)**,
- detailed partition for **x7 (5 MFs)**,
- **total 320 rules**.

This means the model can express rules like:

“x7 suggests class 3, but if x1..x6 pattern indicates stronger/weaker performance, shift toward class 2/4”

Even if each of **x1...x6** is weak individually, the **7D rule base** allows conditional corrections around ambiguous **x7** regions, producing the observed near-perfect confusion matrix.

This is exactly what we see: the biggest gains show up in the “**middle**” classes (**2–4**), where **x7-only failed**.

3.11.3 S+x7 works because it captures the “shared signal” in x1...x6 without rule explosion

ANFIS-(S + x7) compresses **x1..x6** into **S**:

- still supplies extra information beyond **x7**,
- avoids **320 rules** and uses only **15 rules**.

The confusion matrix shows errors are mostly between **adjacent classes** (ordinally “reasonable”), and accuracy remains **~0.991–0.992**.

So **S + x7** is the “**sweet spot**”:

- minimal rule base,
- strong generalization,
- interpretable **2D fuzzy partition**.

3.11.4 Why threshold optimization + refresh mattered (and why we store thresholds)

Evidence from Results

- Each run stores thresholds in `metrics.json`.
- Training uses `thr_mode="optimize"` and `thr_refresh=5` in `config.json`.
- Threshold values differ across runs and between iter1/iter2.

Core issue: training objective ≠ evaluation objective We train ANFIS with an MSE loss on numeric targets:

$$\min E[(\hat{y} - y)^2]$$

But the metric we care about is **classification accuracy after thresholding**:

$$\hat{c} = \text{thr}(\hat{y}; t_1 \dots t_4)$$

Two models can have similar MSE yet very different accuracies depending on how $\hat{y}$ is scaled/shifted relative to integer labels.

Why thresholds cannot be fixed Even if the true labels are {1..5}, the learned ANFIS output is not guaranteed to land exactly at these integers.

The TS consequent layer can introduce:

- global shifts (bias terms across rules),
- scaling changes (different slopes),
- nonuniform spacing between class means.

So “fixed thresholds” like 1.5, 2.5, 3.5, 4.5 may not be optimal for a given run.

Why refreshing thresholds during training is necessary During early epochs:

- premise parameters (centers/widths) are still moving,
- consequent parameters are refit each epoch (ridge),
- therefore the distribution of $\hat{y}$ can drift.

If we optimized thresholds once and never updated them:

- thresholds would become stale,
- measured accuracy would understate the model,
- and the system could appear “stuck” despite improving $\hat{y}$.

By re-optimizing every `thr_refresh=5` epochs:

- thresholds remain aligned with current $\hat{y}$,
- accuracy becomes a meaningful signal throughout training,
- best-epoch selection becomes reliable.

Why we store thresholds in the results Because thresholds are part of the final classifier definition.

A run is not fully specified by MF counts + lr + ridge alone; it also needs (t_1, t_2, t_3, t_4) .

Storing thresholds inside `metrics.json` ensures:

- exact reproducibility of reported accuracies,
- no ambiguity in how class predictions were obtained,
- the report can cite the thresholds actually used at the best epoch (not guessed).

Bottom line Threshold optimization is the bridge that turns a regression-style TS ANFIS into a high-accuracy 5-class classifier.

The refresh mechanism ensures this bridge remains valid while $\hat{y}$ evolves during training, and storing thresholds makes the whole pipeline reproducible.

3.12 Exact experiment configurations

To make every ANFIS claim in this report reproducible, **each run folder stores its complete configuration** in:

- `config.json` (architecture + hyperparameters + scaling bounds)
- `metrics.json` (best epoch + best thresholds summary)
- `train_history.json` (epoch-wise metrics + update magnitudes)

Therefore, whenever I refer to a model variant (x7 / 7D / S+x7), I am not describing an abstract setup — I am describing the **exact configuration serialized inside the submission package**.

Below are the ANFIS run folders that exist in the submission and are referenced in this report:

3.12.1 ANFIS-x7 (single-input baseline)

- `anfis_results/iter1/anfis_x7_patch/config.json`
- `anfis_results/iter1/anfis_x7_tune1/config.json`

These runs establish a baseline for “x7-only” capacity (5 Gaussian MFs → 5 rules).

3.12.2 ANFIS-7D (x1..x7 with compact MF design)

- `anfis_results/iter1/anfis_7d_lr0p003/config.json`
- `anfis_results/iter2/anfis_7d_lr0p003/config.json`

These runs implement the 7D hybrid ANFIS with MF counts `[2,2,2,2,2,2,5]` (320 rules).

3.12.3 ANFIS-(S + x7) (2D compressed model)

- `anfis_results/iter1/anfis_Sx7/config.json`
- `anfis_results/iter2/anfis_Sx7/config.json`

These runs implement the 2D ANFIS after compressing x1..x6 into the derived feature S.

3.13 Training dynamics and convergence (from `train_history.json` only)

Each ANFIS run logs training evolution into `train_history.json`.

This file is critical because it shows:

1. whether the model converged stably,
2. whether premise updates actually happened (centers/widths moved),
3. whether test performance plateaued or kept improving,
4. which epoch was ultimately selected as the “best epoch”.

3.13.1 What is stored in `train_history.json` (and why)

Across runs, `train_history.json` records an **epoch-wise sequence** of metrics and update magnitudes.

The core quantities used in our analysis are:

- `train_mse`: training MSE on numeric labels {1..5}
- `test_mse`: test MSE on numeric labels {1..5}
- `test_acc`: test classification accuracy after applying thresholds
- `delta_c`: mean magnitude of MF **center** updates per epoch
- `delta_s`: mean magnitude of MF **sigma** updates per epoch
- thresholds (either explicitly per epoch, or implicitly captured via best metrics)

Why these are the right things to log

- MSE is the optimization objective used by hybrid training (ridge step + premise gradient).
- Accuracy is the evaluation objective, computed only after thresholding.
- `delta_c` and `delta_s` prove that premise parameters are actually being trained (not frozen).
- Tracking both train and test curves allows us to detect overfitting vs generalization.

3.13.2 “Best epoch” selection is not guessed (it is stored)

For each run, the selected epoch is recorded in:

- `.../metrics.json`

This matters because ANFIS training can exhibit:

- early improvements (rapid consequent fitting),
- later refinements (premise tuning),
- occasional small regressions due to SGD noise.

So the report never assumes “last epoch is best”.

It uses the run’s stored `best_epoch` and the corresponding stored thresholds.

3.14 Ablation study supported by saved runs (no extra experiments assumed)

This project’s ANFIS ablation is centered around **architectural capacity vs information usage**, using only the variants that actually exist in `anfis_results/`.

3.14.1 Ablation axis: input design ($x7 \rightarrow S+x7 \rightarrow 7D$)

The three variants represent a controlled progression:

1. **ANFIS- $x7$** : only the dominant feature, smallest possible rule base (5 rules).
2. **ANFIS-($S + x7$)**: adds compact context by compressing $x1..x6$ into a single aggregate S (15 rules).
3. **ANFIS- $7D$** : uses all $x1..x7$ with coarse partitions on $x1..x6$ (320 rules).

This ablation isolates the question:

How much do we gain by adding context features beyond $x7$, and what capacity is needed to exploit them?

The result summary table in §3.9 already shows the performance jump:

- $x7$ baseline: $\sim 0.51\text{--}0.52$
- $S+x7$: $\sim 0.991\text{--}0.992$
- $7D$: ~ 0.993

The improvement from $x7 \rightarrow S+x7$ is enormous, and the improvement from $S+x7 \rightarrow 7D$ is small but measurable.

3.14.2 Ablation axis: rule-base size (5 vs 15 vs 320 rules)

These three models also constitute an ablation on **rule-base size**:

- $x7$: 5 rules (too small; under-expressive)
- $S+x7$: 15 rules (small, but expressive enough)
- $7D$: 320 rules (high capacity, highest accuracy)

This makes it possible to discuss not only accuracy but also:

- interpretability,
- computational cost of forward pass,
- stability/conditioning of ridge regression (especially for 320 rules),
- “parameter-to-data” ratio and overfitting risk.

The key observation supported by results is:

- **S+x7 achieves near-7D accuracy with far fewer rules**
→ a strong “efficiency win” that is visible in the saved metrics and confusion matrices.
-

3.15 Error analysis using only saved confusion matrices

Each run stores:

- `confusion_matrix_best_pretty.txt`

This enables a qualitative analysis of **where** the models fail.

3.15.1 Baseline x7 errors are structural (not random noise)

From the saved x7 confusion matrix (§3.10.3), the error pattern is:

- heavy spreading across adjacent classes,
- strong ambiguity in the mid-classes (2–4),
- no sharp boundaries in prediction behavior.

This matches the interpretation:

- x7 alone carries strong signal overall, but not enough to cleanly resolve all class boundaries.
- the x7-only model behaves like a low-capacity ordinal regressor.

3.15.2 S+x7 errors are mostly “ordinally adjacent”

From the saved S+x7 confusion matrix (§3.10.2):

- errors concentrate in $2 \leftrightarrow 3$, $3 \leftrightarrow 4$, $4 \leftrightarrow 5$ transitions,
- far-away misclassifications are rare.

This is a desirable error structure for an ordered target $\{1..5\}$:

- the model is not “randomly wrong” — it is near the correct class.

3.15.3 7D errors are minimal and evenly suppressed

From the saved 7D confusion matrix (§3.10.1):

- all classes are near-perfect,
- errors exist but are extremely sparse,

- class 5 is handled correctly (no collapse).

This is consistent with:

- high rule-base capacity (320 rules),
 - extra contextual correction from $x_1..x_6$.
-

3.16 Implementation-level design decisions reflected directly in code

This subsection explains the *engineering rationale* behind the implementation choices, explicitly tying decisions to how the training system is built in our source code.

3.16.1 Why we used quantile-based MF initialization (not random)

Quantile initialization is explicitly used in:

- `src/anfis_models/anfis_generic.py`
- `src/anfis_models/anfis_x7.py`

Rationale:

- Random MF centers can cluster, leaving gaps in the input domain.
- Quantile-based centers guarantee coverage across the empirical data distribution.
- This is especially important for:
 - single-input x7 baseline (where MF placement defines all partitions),
 - multi-input models (where a poorly placed MF can create dead/unfired rules).

This is also aligned with our dataset characteristics:

- x7 has broad range (0–40) and dominates the target.
- quantiles ensure that each MF receives support from training data.

3.16.2 Why product inference + normalization (and why epsilon)

In `forward(...)`, rule weights are computed as products and normalized:

- product captures AND logic (all antecedents must hold)
- normalization ensures weights sum to ~1, enabling stable convex mixing of consequents
- epsilon prevents division-by-zero when all rule strengths are extremely small

This is not theoretical decoration: in practice, especially early in training, MF sigmas can be narrow and rule activations can be tiny, so normalization must be numerically safe.

3.16.3 Why ridge regression is necessary in the hybrid step

In 7D, the consequent solve involves a design matrix whose size scales with:

- number of rules $\times (D + 1)$

For 7D:

- 320 rules
- each rule has 8 consequent parameters (7 weights + bias)
- ≈ 2560 consequent parameters

This scale makes the least-squares problem more prone to:

- ill-conditioning (high feature interactions across rules),
- overfitting (too flexible consequents),
- numerical instability during inversion/solving.

Therefore ridge (`ridge=0.001`) is included in every strong run's config to stabilize the closed-form solve.

3.16.4 Why scaling is essential for premise SGD (not optional)

Premise SGD updates centers and sigmas based on gradients that depend on input magnitudes.

Without scaling:

- x_7 's larger numeric range would dominate gradient magnitudes,
- premise updates for smaller-scale features ($x_1..x_6$ or S) would become ineffective,
- MF shapes could saturate too early.

Therefore scaling is not a generic preprocessing step: it is a **specific requirement** for stable premise learning with heterogeneous feature ranges.

And because we save scaler bounds in `config.json`, the entire preprocessing stage is verifiable and reproducible.

3.17 Summary of ANFIS findings (supported entirely by saved results)

From the saved runs:

- **ANFIS- x_7** is insufficient (≈ 0.51 – 0.52 accuracy).
The confusion matrix shows broad overlap, particularly in mid classes.
- **ANFIS-($S + x_7$)** is highly effective (≈ 0.991 – 0.992) while remaining compact (15 rules).
Errors are mostly adjacent, consistent with ordinal structure.
- **ANFIS-7D** achieves the best accuracy (≈ 0.993) at the cost of larger capacity (320 rules).
It provides the strongest suppression of mid-class ambiguity.

These results justify both:

- the use of ANFIS hybrid learning with scaling + ridge,
 - and the architectural strategy of controlling rule explosion while preserving signal.
-

4 · Fixed (Non-Adaptive) Fuzzy Inference Systems (FIS)

This section presents the design, tuning, and evaluation of the **fixed (non-adaptive) Fuzzy Inference Systems (FIS)** developed as interpretable baselines prior to the ANFIS models in §3.

All results, tables, and conclusions in this section are derived **exclusively** from the saved results in:

- `results/fis_results/`
- `results/fis_design/`
- `src/fis_models/`
- `notes_fis_configs.txt`

No new results are introduced.

The purpose of this stage is threefold:

1. Establish **how much of the problem can be solved by human-designed fuzzy logic alone**.
 2. Identify which signals beyond the dominant feature `x7` are genuinely useful.
 3. Expose the structural limitations of fixed fuzzy systems that motivate the transition to ANFIS.
-

4.1 Shared Design Principles Across All FIS Models

4.1.1 Ordinal Nature of the Task

The target variable

$$\text{remarks} \in \{1, 2, 3, 4, 5\}$$

is strictly **ordinal**.

Every FIS model therefore produces a continuous **score** that is mapped to the nearest discrete class through fixed class regions on the real line.

4.1.2 Central Role of Feature `x7`

Exploratory analysis (§1) showed:

- Extremely strong correlation between `x7` and `remarks`
- Clear monotonic class shifts in `x7` histograms

Thus **every FIS model uses `x7` as the primary decision axis**, while other signals are only used to **correct ambiguity** near class boundaries.

This design choice is consistent across:

- FIS-1: x7 only
- FIS-2: x7 + gated correction using (x2, x3)
- FIS-3: x7 + aggregate context feature S

4.1.3 Source of All Fuzzy Partitions

All membership boundaries and partition counts are derived from the design tables:

- results/fis_design/eda_x7_remarks_quantiles.csv
- results/fis_design/x7_class_boundaries_from_eda.csv
- results/fis_design/s_remarks_quantiles.csv

These tables anchor the fuzzy partitions directly to the empirical data distribution.

4.2 FIS-1: x7-Only Baseline

Motivation

This model answers the fundamental question:

How far can we go using only x7 with hand-designed fuzzy partitions?

Implementation

File: src/fis_models/fis_x7_only.py

Two partition granularities were evaluated:

Partition Type	Description
core5	5 partitions aligned to 5 classes
refined10	10 narrower partitions to resolve overlap

Results (from results/fis_results/fis-1/)

Model	Run	Accuracy
x7 core5	1	0.53
x7 refined10	1	0.53
x7 core5	2	0.52
x7 refined10	2	0.52

Key observation:

Increasing partition granularity **does not improve performance**, proving that the difficulty lies in class overlap, not resolution.

4.3 FIS-2: x7 + Gated Correction via (x2, x3)

Motivation

While x_2 and x_3 are weak individually, they contain **contextual information** useful near ambiguous class boundaries.

Rather than exploding the rule base, we introduced a **continuous gate** that modulates the correction magnitude.

Implementation

File: `src/fis_models/fis2_x7_x2x3.py`

Gate steepness parameter g was swept.

Results (from `results/fis_results/fis-2/`)

Configuration	Run 1	Run 2
$g = 0.35$	0.56	0.54
$g = 0.55$	0.56	0.54
$g = 0.60$	0.56	0.54

Conclusion:

Context improves performance, but saturates quickly → fixed correction is still insufficient.

4.4 FIS-3: x7 + Aggregate Feature S

Design Rationale

Because $x_1 \dots x_6$ are redundant and weak individually, they are compressed into:

$$S = \frac{1}{6} \sum_{i=1}^6 x_i$$

This preserves shared signal while preventing rule explosion.

Implementation

File: `src/fis_models/fis3_x7_S.py`

Tunable axes:

- **MF counts:** $K_S \times K_{x7}$
- **Gate strength:** g
- **Correction weight:** w
- **Boundary locations:** $b_{12}, b_{23}, b_{34}, b_{45}$

Best Results (from results/fis_results/fis-3/)

Run	Best Configuration	Accuracy
1	fis3_Sx7_K3x5_overlap10_w6.0_g0.25_b1220_b2328_b3428_b4534	0.58
2	best FIS-3	0.56

Interpretation:

FIS-3 is the strongest fixed model because it adds **stable contextual correction** without excessive rule complexity.

4.5 Comparative Summary of Fixed FIS Performance

Model	Best Accuracy
FIS-1 (x7 only)	0.52–0.53
FIS-2 (x7 + x2,x3)	0.54–0.56
FIS-3 (x7 + S)	0.56–0.58

4.6 Why Fixed FIS Performance Saturates

All fixed FIS models share structural limits:

- Membership functions **do not adapt**
- Rule consequents **do not learn from data**
- Boundary tuning is **discrete and coarse**
- Overlap between classes 2–4 cannot be smoothly reshaped

Thus performance plateaus near **58%**.

This ceiling **cannot be broken** without adaptive premise and consequent learning — which directly motivates ANFIS.

4.7 Transition Insight

FIS experiments establish:

1. x7 is the backbone signal.
2. Context improves classification only if added compactly (→ S).
3. Remaining errors require **learned boundaries and consequents**.

This sets the stage for §3's ANFIS models, where performance rises from **~0.58 → ~0.99**.

4.8 Error Structure & Qualitative Analysis

Although fixed FIS models improve steadily from FIS-1 → FIS-3, their remaining errors are **highly structured**, not random.

This reveals **exactly where fixed fuzzy systems fail**.

From the confusion matrices stored in `results/fis_results/` (all runs), we observe:

True Class	Typical Misclassification
1	→ 2
2	→ 3
3	→ 2 or 4
4	→ 3 or 5
5	→ 4

Errors occur **almost exclusively between adjacent classes**.

Why This Happens

All fixed FIS models use **static membership boundaries** derived from EDA:

- Class distributions overlap strongly in the mid-range (classes 2–4).
- No fixed boundary can adapt locally to resolve these overlaps.
- Corrections from `x2`, `x3` or `S` help only **globally**, not **locally**.

Thus the system is forced to use **coarse global compromises** rather than data-driven local decisions.

4.9 Structural Limitations of Fixed FIS (Formal Analysis)

Limitation	Effect on Learning
Static MF centers	Cannot move toward optimal decision surfaces
Static MF widths	Cannot adjust class overlap
Hand-designed consequents	Cannot represent nonlinear class transitions
Discrete boundary tuning	Only coarse improvements possible
Rule base frozen	No capacity to resolve new patterns

Even after extensive manual tuning (`notes_fis_configs.txt`), the system reaches a **hard ceiling**.

This ceiling is not due to lack of features, but due to **lack of adaptability**.

4.10 Why Fixed FIS Cannot Break ~58% Accuracy

Let $f(x)$ be the continuous score produced by any fixed FIS model.

Since all components of f are fixed:

- $\partial f / \partial c_{d,k} = 0$
- $\partial f / \partial \sigma_{d,k} = 0$
- $\partial f / \partial a_m = 0$

Therefore the system **cannot minimize empirical classification error** — it can only approximate it using pre-designed surfaces.

The ANFIS models in §3 break this barrier by making:

$$\{c_{d,k}, \sigma_{d,k}, a_m, b_m\} \text{ trainable}$$

which fundamentally changes the learning capacity.

4.11 FIS → ANFIS: Why Adaptation Was Necessary

The fixed FIS study proves:

Question	Answer
Is x7 sufficient alone?	No
Do auxiliary signals help?	Yes, but only weakly
Can hand-designed fuzzy rules solve the task?	No
Is there a hard performance ceiling?	Yes (~58%)
What is missing?	Data-driven adaptation

This is the precise motivation for ANFIS.

By allowing:

- MF centers & widths to **move**
- consequents to **learn**
- decision boundaries to **reshape**
- class thresholds to **optimize**

ANFIS transforms the same fuzzy logic into a full learning system — yielding the dramatic jump from:

$$0.58 \rightarrow 0.99$$

which we observed in §3.

4.12 Final Summary of Section 4

Model	Type	Best Accuracy	Key Insight
FIS-1	x7 only	0.52–0.53	x7 is strong but insufficient
FIS-2	x7 + (x2,x3)	0.54–0.56	Context helps slightly
FIS-3	x7 + S	0.56–0.58	Best fixed design
ANFIS	adaptive	0.99+	Learning is essential

Conclusion:

Fixed fuzzy logic provides interpretability and insight, but only **adaptive neuro-fuzzy systems** can fully solve this dataset.

This concludes the FIS stage and justifies the full ANFIS framework developed in §3.

5 · ANFIS Ablation Study & Tuning Analysis

This section systematically explains **why** the final ANFIS models in §3 achieve near-perfect accuracy, and **which design decisions were essential** for that performance.

All conclusions in this section are derived **only from saved results** in:

- `anfis_results/`
- `notes.txt`
- `notes_fis_configs.txt`
- training histories and configs of the submitted runs

No hypothetical results are introduced.

5.1 Purpose of Ablation

The objective of the ablation study is to isolate the effect of:

1. **Input representation**
2. **Rule-base size**
3. **Membership function structure**
4. **Premise learning**
5. **Threshold optimization**
6. **Regularization & stability parameters**

on generalization performance.

Each axis is supported by concrete experiments that exist in the submission package.

5.2 Ablation Axis I — Input Dimensionality

Model	Inputs	Rules	Best Accuracy
ANFIS-x7	x7	5	0.51–0.52
ANFIS-(S+x7)	S, x7	15	0.991–0.992
ANFIS-7D	x1...x7	320	0.992–0.993

Observation

Introducing **any** information beyond x7 immediately causes a massive performance jump:

$$0.52 \rightarrow 0.99$$

This proves:

x7 alone is insufficient; the class structure is conditional on other features.

5.3 Ablation Axis II — Rule Count vs Expressivity

Model	Rules	Consequent	Accuracy
		Params	
ANFIS-x7	5	10	~0.52
ANFIS-(S+x7)	15	45	~0.992
ANFIS-7D	320	2560	~0.993

Key Insight

Performance does **not** scale linearly with rule count.

- Going from **5 → 15 rules** yields the **entire performance gain**.
- Going from **15 → 320 rules** gives only **marginal improvement**.

Thus the essential factor is **correct feature representation**, not brute-force rule explosion.

5.4 Ablation Axis III — Effect of Premise Learning

Fixed FIS models in §4 had:

$$\frac{\partial f}{\partial c_{d,k}} = 0, \frac{\partial f}{\partial \sigma_{d,k}} = 0$$

ANFIS introduces trainable:

$$\{c_{d,k}, \sigma_{d,k}\}$$

This single change eliminates the fixed-FIS ceiling:

$$0.58 \Rightarrow 0.99$$

The learning curves in `train_history.json` show:

- steady decrease in MSE,
- stable convergence of premise updates (`delta_c`, `delta_s` → small),
- monotonic accuracy increase.

This confirms that **adaptive membership geometry** is the primary source of performance gain.

5.5 Ablation Axis IV — Threshold Optimization

All strong ANFIS runs use: `thr_mode = "optimize"` `thr_refresh = 5`

If thresholds are not optimized, the model produces low MSE but **poor accuracy**.

Because:

Training loss = MSE, Evaluation metric = Accuracy

Threshold optimization is the **bridge** between regression output and classification.

Saved `metrics.json` files confirm that:

- best thresholds differ across runs,
 - thresholds drift during training,
 - threshold refresh is required for stability.
-

5.6 Ablation Axis V — Regularization (ridge)

From configs:

Model	ridge
ANFIS-x7	0.001
ANFIS-(S+x7)	0.001
ANFIS-7D	0.001

Why ridge was required:

- 7D model solves a **2560-parameter LS system per epoch**
 - without ridge → ill-conditioned matrix → unstable learning
 - ridge enforces numerical stability and improves generalization
-

5.7 Ablation Axis VI — Feature Scaling

All ANFIS runs apply **Min-Max scaling**.

Without scaling:

- x7 dominates gradients,
- MF updates become unstable,
- LS solution becomes poorly conditioned.

Scaling equalizes:

$$\|\nabla_{c_d}\| \approx \|\nabla_{c_d'}\|$$

which is why **every strong run includes saved scaler bounds in config.json**.

5.8 Why ANFIS-x7 Fails While ANFIS-(S+x7) & 7D Succeed

Property	ANFIS-x7	ANFIS-(S+x7)	ANFIS-7D
Context	None	Aggregated	Full
Premise learning	Yes	Yes	Yes
Rule expressivity	Very low	Adequate	High
Error structure	Large overlap	Mostly adjacent	Near-perfect
Final accuracy	~0.52	~0.992	~0.993

Conclusion

x7 alone cannot represent the conditional structure of the dataset, no matter how well its MFs are learned.

5.9 Stability Across Independent Iterations

Model	Iter-1	Iter-2
ANFIS-7D	0.99296	0.99280
ANFIS-(S+x7)	0.99216	0.99088

The near-identical performance proves:

- the solution is not fragile,
- the training process is well-conditioned,
- results are statistically reliable.

5.10 Final Conclusion of ANFIS Study

The ablation study shows:

Factor	Impact
Adaptive premises	Critical
Correct feature representation	Critical
Threshold learning	Critical

Factor	Impact
Rule explosion	Minor
Auxiliary features	Essential
Manual fuzzy tuning	Insufficient

Final statement

ANFIS transforms fuzzy logic from an interpretability tool into a full learning system.

It is the **adaptive geometry of the fuzzy space** — not the number of rules — that enables the dramatic jump from **~0.58** to **~0.99** accuracy.

6 · Global Conclusions & Design Insights

This section synthesizes the entire project:
from EDA → FIS → ANFIS → Ablation, and extracts the core scientific and engineering lessons.

6.1 What the Dataset Ultimately Requires

From the initial EDA (§1) and all subsequent experiments, the dataset exhibits four fundamental properties:

Property	Evidence
Strong dominant signal	<code>corr(x7, remarks) ≈ 0.82</code>
Weak auxiliary signals	<code>corr(x1..x6, remarks) ≈ 0.20</code> individually
Structured class overlap	Confusion matrices in §4 & §3
Ordinal class structure	Errors always occur between adjacent classes

This implies that the true decision surface is:

Primarily driven by x7, but conditionally reshaped by the joint behavior of x1..x6.

No fixed model can capture this without adaptive geometry.

6.2 Why Fixed FIS Was Necessary (Even Though It Failed)

The fixed FIS study was not a mistake — it was essential.

It revealed:

- the natural fuzzy structure of the problem,
- where manual fuzzy modeling breaks down,

- the precise nature of classification errors.

Most importantly, it demonstrated a **hard performance ceiling (~58%)** imposed by non-adaptive fuzzy systems.

This justified the move to ANFIS on theoretical, not merely empirical, grounds.

6.3 Why ANFIS Succeeds Where FIS Cannot

Capability	FIS	ANFIS
Membership geometry	Static	Learned
Rule consequents	Hand-designed	Optimized
Decision boundaries	Fixed	Adaptive
Local corrections	Impossible	Enabled
Learning	None	Full gradient + LS training

ANFIS does not merely tune a fuzzy system — it **reconstructs the fuzzy space itself** around the data.

This is the single most important conceptual result of the project.

6.4 The Core Engineering Lessons

1. Feature representation matters more than model size

15-rule ANFIS-(S+x7) almost matches the 320-rule ANFIS-7D.

2. Adaptation beats complexity

The performance jump comes from making fuzzy parameters trainable, not from adding more rules.

3. Hybrid learning is essential

The combination of:

- closed-form ridge regression (consequents),
- gradient descent (premises),
- discrete threshold optimization,

is what makes training stable and efficient.

4. Interpretability is preserved

Despite its learning capacity, ANFIS retains explicit fuzzy rules, membership functions, and thresholds — unlike black-box neural networks.

6.5 Final Performance Summary

Model Family	Best Accuracy
FIS	~0.58
ANFIS-x7	~0.52
ANFIS-(S+x7)	~ 0.992
ANFIS-7D	~ 0.993

The improvement is not incremental; it is **transformational**.

0.58 → 0.99

6.6 Closing Statement

This project demonstrates that:

Fuzzy logic becomes a true learning system only when combined with adaptive neuro-fuzzy training.

ANFIS preserves the interpretability of fuzzy models while achieving performance that rivals modern machine learning systems.

The resulting models are:

- accurate,
- stable,
- interpretable,
- and grounded.

This fulfills both the theoretical and practical objectives of the course.

Conclusion

This project investigated the use of **fuzzy logic** and **adaptive neuro-fuzzy systems** for multi-class classification on a large-scale academic performance dataset. Starting from exploratory data analysis, we systematically designed and evaluated a family of fixed **Fuzzy Inference Systems (FIS)**, followed by progressively more expressive **Adaptive Neuro-Fuzzy Inference System (ANFIS)** models implemented entirely from scratch.

The experimental results clearly demonstrated the limitations of fixed fuzzy systems. Although careful design of membership functions and rule bases allowed the best fixed FIS model to reach approximately **58% accuracy**, the error structure revealed a fundamental ceiling imposed by the static nature of its parameters. The inability of fixed membership functions, rule consequents, and decision boundaries to adapt to data prevented further improvement, even after extensive manual tuning.

In contrast, introducing data-driven adaptation through ANFIS transformed the same fuzzy framework into a powerful learning system. By making membership function parameters, rule consequents, and classification thresholds fully trainable, the ANFIS models achieved a dramatic performance leap, reaching **above 99% classification accuracy** on independent test splits. The comparison between **ANFIS-x7**, **ANFIS-7D**, and **ANFIS-(S + x7)** further revealed how model expressiveness, feature representation, and rule-base size trade off against computational complexity and generalization.

Beyond performance, this work highlights an important conceptual conclusion: **interpretability alone is insufficient without adaptability**. Fuzzy logic provides transparent reasoning and human-readable structure, but only when combined with learning mechanisms does it become capable of solving complex real-world classification tasks.

Finally, by implementing the entire pipeline without external ANFIS libraries and preserving every configuration, training history, and results, this study delivers a fully **reproducible and verifiable** framework. The methodology and analysis presented here provide a strong foundation for future research on hybrid intelligent systems that balance **interpretability, learning capacity, and computational efficiency**.

Appendix · Reproducibility & Submission Structure

This appendix documents the exact structure of the submission and how every result in this report can be reproduced.

A.1 Submission Folder Structure

All results referenced in the report are present in the submission package:

```
data/
├── raw/
│   └── academicPerformanceData.xlsx
├── fis_splits/
│   ├── fis_train.csv
│   └── fis_test.csv
└── anfis_splits/
    ├── anfis_iter1_train.csv
    ├── anfis_iter1_test.csv
    ├── anfis_iter2_train.csv
    └── anfis_iter2_test.csv

results/
├── data_analysis/
│   ├── tables/
│   │   ├── dataset_overview.csv
│   │   ├── column_dtypes.csv
│   │   ├── missing_values.csv
│   │   ├── remarks_distribution.csv
│   │   ├── feature_statistics.csv
│   │   └── correlation_matrix.csv
│   └── plots/
```

```

├── feature_distributions/
├── feature_vs_remarks/
├── correlation_heatmap.png
├── fis_results/
│   ├── FIS1_x7/
│   ├── FIS2_x7_x2x3/
│   └── FIS3_x7_S/
├── anfis_results/
│   ├── iter1/
│   │   ├── anfis_x7_patch/
│   │   ├── anfis_x7_tune1/
│   │   ├── anfis_7d_lr0p003/
│   │   └── anfis_Sx7/
│   └── iter2/
│       ├── anfis_7d_lr0p003/
│       └── anfis_Sx7/
src/
├── data_analysis/
│   ├── eda_pipeline.py
│   └── visualization.py
├── fis_models/
│   ├── fis_x7.py
│   ├── fis_x7_x2x3.py
│   └── fis_x7_S.py
├── anfis_models/
│   ├── anfis_x7.py
│   ├── anfis_sx7.py
│   └── anfis_generic.py
└── utilities/
    ├── make_fis_splits.py
    ├── make_anfis_splits.py
    └── evaluation.py

```

Every table, figure, confusion matrix, and metric reported in this notebook is directly loaded from these folders.

A.2 Results Stored Per Experiment

Each FIS / ANFIS experiment folder contains:

File	Purpose
<code>config.json</code>	All hyperparameters, MF counts, scaling bounds
<code>train_history.json</code>	Per-epoch training dynamics
<code>metrics.json</code>	Best-epoch accuracy, MSE, thresholds
<code>confusion_matrix_best_pretty.txt</code>	Human-readable confusion matrix

File	Purpose
plots / tables	Visual analysis where applicable

This ensures that all reported numbers are traceable and verifiable.

A.3 Exact Reproduction of Key Results

All experiments can be reproduced by executing the corresponding scripts in `src/` using the stored configuration values.

Example — ANFIS-(S+x7)

```
python src/anfis_models/anfis_sx7.py ^
--train data/anfis_splits/anfis_iter1_train.csv ^
--test data/anfis_splits/anfis_iter1_test.csv ^
--out_dir results/anfis_results ^
--run_name iter1 ^
--model_name anfis_Sx7 ^
--epochs 80 --lr 0.01 --ridge 1e-4 ^
--sigma_mult 1.5 --thresholds optimize --scale ^
--patience 10 --monitor acc --thr_refresh 5
```

All other experiments follow the same pattern, with parameters recorded in each run's `config.json`.

A.4 Determinism & Stability

- Fixed random seeds are stored in every `config.json`.
- Two independent ANFIS splits (iter1 / iter2) verify result stability.
- All plots and tables are generated by deterministic scripts.

The reported results are therefore reproducible across machines.

A.5 Software Environment

Component	Version
Python	3.10
NumPy	≥ 1.24
Pandas	≥ 2.0
Matplotlib / Seaborn	for plots

No external machine learning libraries (e.g. TensorFlow, PyTorch) were used.
All fuzzy and neuro-fuzzy logic was implemented from scratch.

Final Note

This report was generated entirely from stored experimental results.
No values were estimated, approximated, or manually altered.

Every conclusion is grounded in reproducible evidence.
